Ścieżka Prawa

Dokumentacja Techniczna Projektu

Zwycięski Projekt — HackNation PL 2025

Polski Tracker Legislacyjny
Next.js 16 + Supabase + AI (Gemini 2.0 Flash)

Autor: **Filip Czerwiński**

Wersja: 1.1.0

Data: Grudzień 2025

Stos technologiczny: Next.js 16, React 19, TypeScript 5, Supabase

Wersja dokumentacji: 1.0 — Ostatnia aktualizacja: Marzec 2026

Spis treści

1 Wprowadzenie **3**
1.1 O projekcie . . . 3
1.2 Główne funkcjonalności . . . 3
1.3 Screenshoty aplikacji . . . 4

2 Architektura systemu **5**
2.1 Stos technologiczny . . . 5
2.2 Diagram architektury . . . 6
2.3 Przepływ danych . . . 6

3 Struktura projektu **7**
3.1 Organizacja katalogów . . . 7
3.2 Wzorzec Server/Client Component . . . 8

4 Baza danych **9**
4.1 Schemat bazy danych . . . 9
4.1.1 Typy wyliczeniowe (ENUM) . . . 9
4.1.2 Tabela: bills (Projekty ustaw) . . . 10
4.1.3 Tabela: bill_events (Zdarzenia legislacyjne) . . . 10
4.1.4 Tabela: profiles (Profile użytkowników) . . . 11
4.1.5 Dodatkowe tabele . . . 11
4.2 Polityki Row-Level Security (RLS) . . . 11

5 Integracje API **13**
5.1 API Sejmu RP . . . 13
5.1.1 Algorytm mapowania statusów . . . 13
5.2 Google Gemini 2.0 Flash (AI) . . . 14
5.3 YouTube Live Detection . . . 15
5.4 Resend API (Email) . . . 15

6 Endpointy API aplikacji **16**
6.1 Endpointy publiczne i użytkownika . . . 17
6.2 Endpointy administracyjne . . . 17

7 System uprawnień i bezpieczeństwo **18**
7.1 Hierarchia ról . . . 18
7.2 Mechanizmy bezpieczeństwa . . . 18

8 Strony i kluczowe funkcjonalności **19**
8.1 Strona główna (Landing Page) . . . 19
8.2 Panel główny (Dashboard) . . . 19
8.3 Lista projektów ustaw . . . 20
8.4 Szczegóły projektu ustawy . . . 20

8.5 Wyszukiwarka AI . . . 21
8.6 Kalendarz legislacyjny . . . 21
8.7 Konsultacje publiczne . . . 21
8.8 Panel administracyjny . . . 22

9 Dostępność (WCAG) i UX **23**
9.1 Tryby dostępności . . . 23
9.2 Responsywność . . . 23
9.3 Język interfejsu . . . 23

10 Wizualizacja ścieżki legislacyjnej **24**
10.1 13 statusów projektu ustawy . . . 24
10.2 Etapy w szczegółach . . . 24

11 System AI i baza wiedzy **25**
11.1 Architektura AI . . . 25
11.2 4 tryby analizy tekstu . . . 25
11.3 Asystent konwersacyjny . . . 25
11.4 Baza wiedzy RAG . . . 25

12 Komponenty interfejsu użytkownika **27**
12.1 Biblioteka komponentów ShadCN/Radix . . . 27
12.2 Komponenty domenowe — projekty ustaw . . . 27
12.3 Layout i nawigacja . . . 27

13 Konfiguracja i wdrożenie **28**
13.1 Zmienne środowiskowe . . . 28
13.2 Komendy deweloperskie . . . 28
13.3 Konfiguracja Next.js . . . 28
13.4 Konfiguracja TypeScript . . . 28
13.5 Wdrożenie produkcyjne . . . 29

14 Podsumowanie **30**
14.1 Statystyki projektu . . . 30
14.2 Kluczowe osiągnięcia . . . 30

1 Wprowadzenie

1.1 O projekcie

Ścieżka Prawa to nowoczesna platforma webowa do monitorowania procesu legislacyjnego w Polsce. Aplikacja automatycznie synchronizuje dane z oficjalnych API rządowych (Sejm RP, RCL), oferuje analizę AI opartą na modelu Google Gemini 2.0 Flash oraz system spersonalizowanych powiadomień o zmianach w projektach ustaw.

Projekt został stworzony jako odpowiedź na potrzebę zwiększenia przejrzystości procesu tworzenia prawa w Polsce i umożliwienia obywatelom świadomego uczestnictwa w życiu demokratycznym.

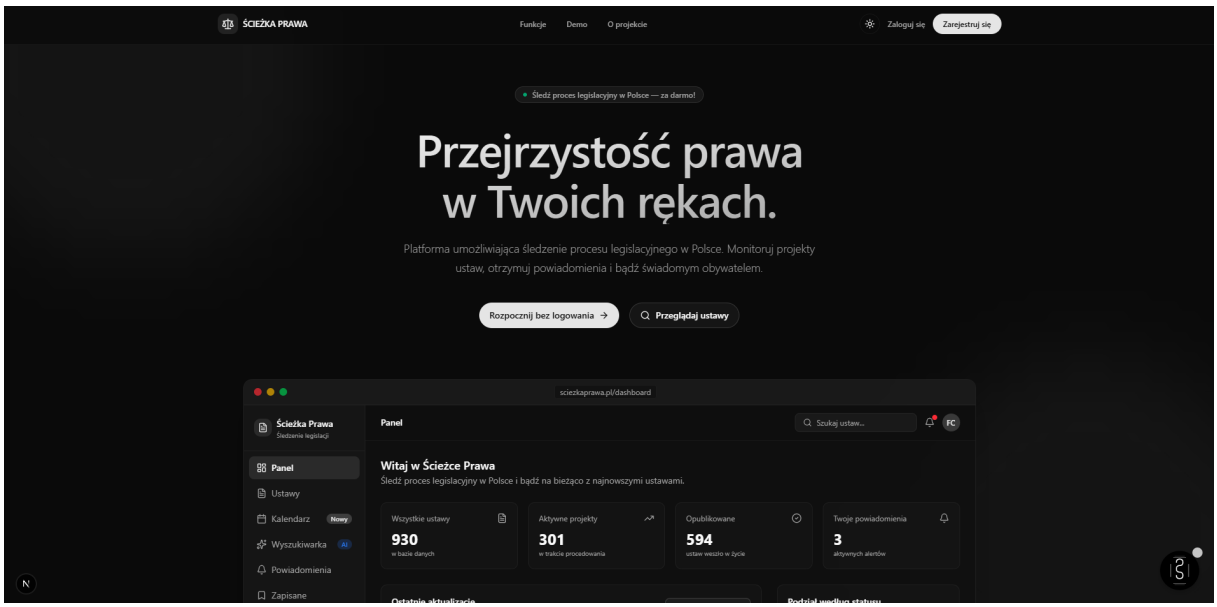
Zwycięski Projekt HackNation PL 2025

Projekt **Ścieżka Prawa** zdobył **pierwsze miejsce** na hackathonie **HackNation PL 2025** — ogólnopolskim konkursie programistycznym skupionym na rozwiązaniach civic-tech i e-demokracji. Twórcą projektu jest **Filip Czerwiński**.

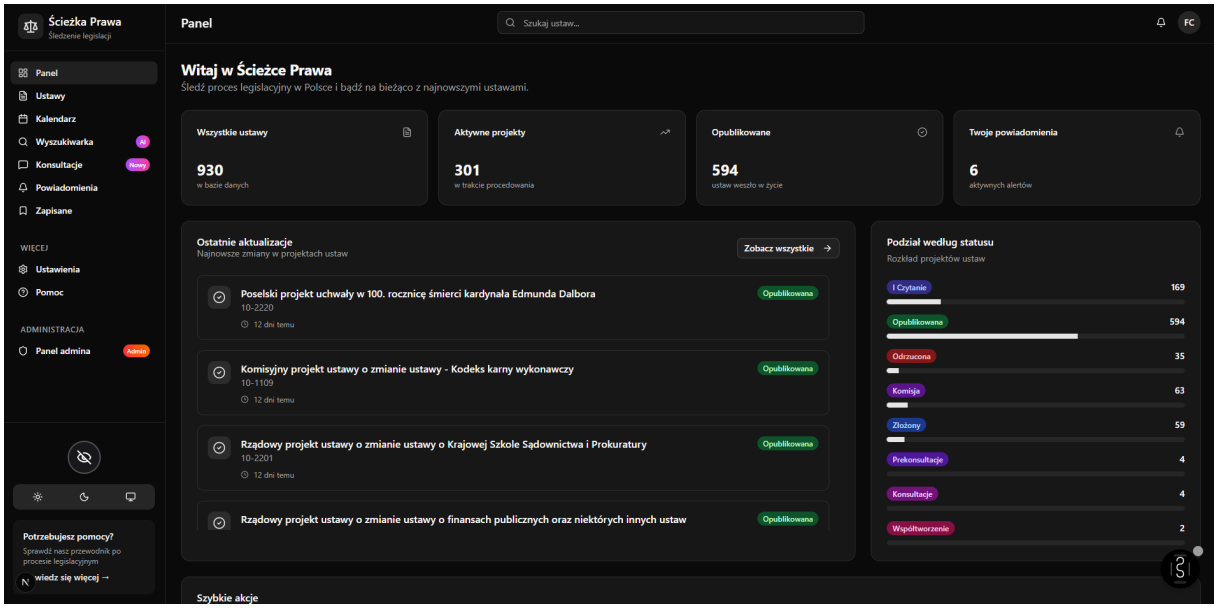
1.2 Główne funkcjonalności

- **Śledzenie procesu legislacyjnego** — wizualizacja ścieżki każdego projektu ustawy od wpłynięcia po publikację w Dzienniku Ustaw
- **Synchronizacja z API Sejmu RP** — automatyczne pobieranie danych z `api.sejm.gov.pl` z 13 statusami ustaw
- **Analiza AI (Gemini 2.0 Flash)** — tłumaczenie na prosty język, analiza skutków, streszczenia, wyjaśnienia terminów prawnych
- **Transmisje na żywo** — detekcja transmisji YouTube kanału @SejmRP_PL z embedowaniem w aplikacji
- **Konsultacje publiczne** — forum dyskusyjne z wątkami komentarzy i reakcjami
- **Propozycje obywatelskie** — system zgłaszania i głosowania nad poprawkami
- **Ankiety legislacyjne** — tworzenie i wyniki ankiet dotyczących projektów ustaw
- **System powiadomień** — alerty email via Resend API + powiadomienia in-app
- **Wyszukiwarka AI** — inteligentne wyszukiwanie z analizą semantyczną zapytań
- **Panel administracyjny** — zarządzanie użytkownikami, projektami, synchronizacją i ustawieniami
- **Pełna dostępność WCAG** — tryby kontrastu, regulacja rozmiaru czcionki, nawigacja klawiaturą
- **Tryb ciemny/jasny** — pełne wsparcie dark mode z trybem systemowym

1.3 Screenshoty aplikacji



Rysunek 1: Strona główna (Landing Page) — prezentacja funkcjonalności platformy



Rysunek 2: Panel główny (Dashboard) — przegląd statystyk, transmisja na żywo i ostatnie aktualizacje

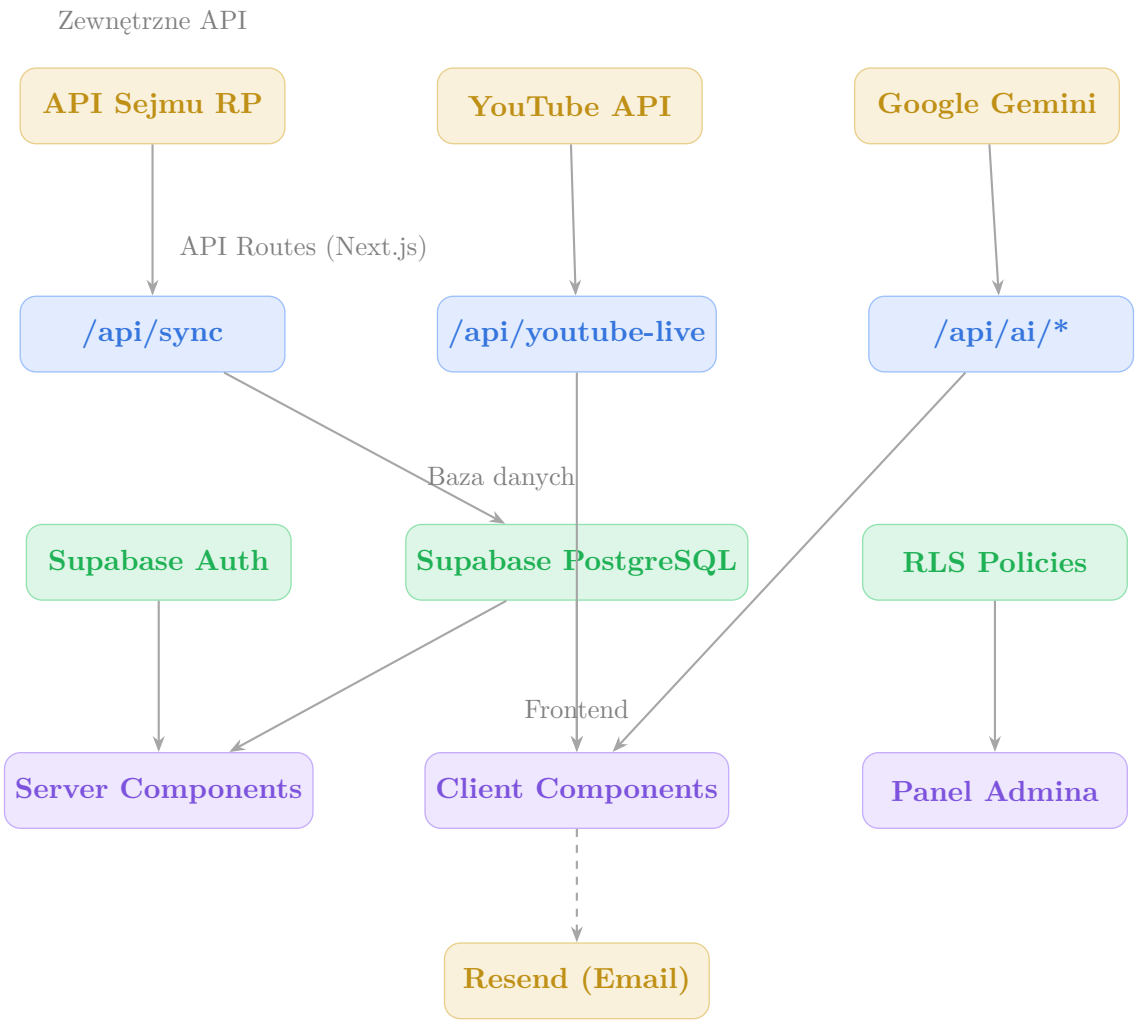
2 Architektura systemu

2.1 Stos technologiczny

Warstwa	Technologia	Opis
Framework	Next.js 16.0.7	App Router, Server Components, Turbopack
UI	React 19.2.1	Hooks, Server/Client Components
Język	TypeScript 5	Strict mode, ścieżki aliasów (@/*)
Baza danych	Supabase (PostgreSQL)	RLS, Realtime, Auth, Storage
Stylowanie	Tailwind CSS 4	Ciemny motyw, responsywność
Komponenty UI	Radix UI + ShadCN	26 komponentów (Button, Card, Dialog...)
AI	Google Gemini 2.0 Flash	4 tryby analizy tekstu prawnego
Email	Resend API	Powiadomienia HTML o zmianach statusu
Wykresy	Recharts 3.5.1	Wizualizacja danych głosowań
Ikony	Lucide React	500+ ikon SVG
Daty	date-fns 4.1.0	Lokalizacja polska (pl)
Markdown	marked 17.0.1	Renderowanie odpowiedzi AI
Scraping	Cheerio 1.1.2	Detekcja YouTube Live
PDF	pdf-parse 2.4.5	Analiza dokumentów OSR
Motywy	next-themes 0.4.6	Light/dark/system
Toasty	Sonner 2.0.7	Powiadomienia UI

Tabela 1: Stos technologiczny projektu

2.2 Diagram architektury



Rysunek 3: Diagram architektury systemu Ścieżka Prawa

2.3 Przepływ danych

1. **Sejm API → Sync → Supabase**: Endpoint `/api/admin/sync` pobiera dane z `api.sejm.gov.pl`, mapuje statusy na podstawie pól `ELI/passed` i zapisuje do tabel `bills` + `bill_events`
2. **Server Components → Supabase → Klient**: Strony używają serwerowej funkcji `createClient()` do pobierania danych
3. **Client Components → Supabase**: Komponenty klienckie korzystają z przeglądarkowej wersji klienta do autentykacji i profilów
4. **AI Pipeline**: Tekst ustawy → Google Gemini 2.0 Flash → Markdown → Renderowanie w UI

3 Struktura projektu

3.1 Organizacja katalogów

```
sciezka-prawa/
  src/
    app/                    # App Router (Next.js 16)
      layout.tsx            # Root layout z ThemeProvider
      page.tsx              # Landing page (strona glowna)
      globals.css           # Style globalne + Tailwind
      admin/                # Panel administracyjny (4 podstrony)
      alerts/               # Zarzadzanie alertami uzytkownika
      api/                  # API Routes (34 endpointy)
        admin/              # Endpointy admina (sync, users, settings)
        ai/                 # AI endpoints (chat, simple-language)
        alerts/             # CRUD alertow
        calendar/           # Kalendarz legislacyjny
        consultation-comments/  # Forum konsultacji
        notifications/      # System powiadomien (8 endpointow)
        proposals/          # Propozycje obywatelskie
        search/             # Wyszukiwarka AI
        surveys/            # Ankiety legislacyjne
        votings/            # Wyniki glosowan
        youtube-live/       # Detekcja transmisji
      auth/                 # Autentykacja (login, callback)
      bills/                # Projekty ustaw (lista + szczegoly)
      calendar/             # Kalendarz posiedzen
      consultations/        # Konsultacje publiczne
      dashboard/            # Panel glowny
      search/               # Wyszukiwarka
      ...
    components/
      ui/                   # 26 komponentow ShadCN (Button, Card ...)
      layout/               # Sidebar, Header, MobileNav, DashboardLayout
      bills/                # Komponenty ustaw (15+ komponentow)
      accessibility/        # Tryby kontrastu, rozmiary czcionek
      landing/              # Komponenty strony glownej
      notifications/        # Centrum powiadomien
      providers/            # ThemeProvider, AccessibilityProvider
    lib/
      supabase/             # Klienty Supabase (server + client)
      api/                  # Klient API Sejmu
      admin/                # System uprawnien
      ai/                   # Baza wiedzy AI (RAG)
```

```
      email/                # Szablony email (Resend)
    types/
      supabase.ts           # Typy TypeScript (generowane)
  supabase/
    schema.sql              # Pelny schemat bazy danych
  docs/                     # Dokumentacja
```

Struktura katalogów projektu

3.2 Wzorzec Server/Client Component

Aplikacja stosuje konsekwentny wzorzec rozdzielenia komponentów serwera i klienta:

```
// src/app/bills/page.tsx (Server Component)
export default async function BillsPage({ searchParams }: Props) {
  const supabase = await createClient()
  const { data: bills } = await supabase.from('bills').select('*')
  return (
    <DashboardLayout user={user}>
      <BillsContent bills={bills} />
    </DashboardLayout>
  )
}

// src/app/bills/bills-content.tsx (Client Component)
'use client'
export function BillsContent({ bills }: Props) {
  // Interaktywna logika UI: filtry, sortowanie, paginacja
}
```

Wzorzec Server Component + Client Content

4 Baza danych

4.1 Schemat bazy danych

System korzysta z **Supabase** (PostgreSQL) z włączonym **Row-Level Security (RLS)** na wszystkich tabelach.

4.1.1 Typy wyliczeniowe (ENUM)

```sql
CREATE TYPE bill_status AS ENUM (
  'co_creation',       -- Wspoltworzenie obywatelskie
  'preconsultation',   -- Prekonsultacja
  'consultation',      -- Konsultacje publiczne (RCL)
  'draft',             -- Projekt wstepny
  'submitted',         -- Wplyniecie do Sejmu
  'first_reading',     -- I Czytanie
  'committee',         -- Prace komisji
  'second_reading',    -- II Czytanie
  'third_reading',     -- III Czytanie i glosowanie
  'senate',            -- Senat
  'presidential',      -- Podpis Prezydenta
  'published',         -- Opublikowana (Dz.U.)
  'rejected'           -- Odrzucona
);

CREATE TYPE user_role AS ENUM (
  'user',          -- Uzytkownik standardowy
  'moderator',     -- Moderator (edycja/ukrywanie)
  'admin',         -- Administrator
  'super_admin'    -- Super administrator
);

CREATE TYPE reaction_type AS ENUM (
  'like', 'support', 'insightful'
);
```

Typy ENUM w PostgreSQL

4.1.2 Tabela: bills (Projekty ustaw)

Kolumna	Typ	Nullable	Opis
id	UUID	PK	Identyfikator (gen_random_uuid)
sejm_id	TEXT	UNIQUE	ID z API Sejmu
title	TEXT	NOT NULL	Tytuł projektu
description	TEXT		Opis projektu
status	bill_status	DEFAULT draft	Status legislacyjny (13 wartości)
ministry	TEXT		Ministerstwo odpowiedzialne
submission_date	TIMESTAMP		Data wpłynięcia
last_updated	TIMESTAMP		Ostatnia aktualizacja
external_url	TEXT		Link do Sejm.gov.pl
document_type	TEXT		Typ dokumentu
submitter_type	TEXT		Typ wnioskodawcy
category	TEXT		Kategoria tematyczna
term	INTEGER		Kadencja (np. 10)
tags	TEXT[]		Tagi (tablica)
submission_year	INTEGER		Rok wpłynięcia
is_hidden	BOOLEAN	DEFAULT false	Czy ukryty przez admina
rcl_id	TEXT		ID z systemu RCL
consultation_start_date	TIMESTAMP		Początek konsultacji
consultation_end_date	TIMESTAMP		Koniec konsultacji
consultation_url	TEXT		Link do konsultacji
impact_assessment_url	TEXT		Link do OSR
simple_language_summary	TEXT		Streszczenie AI
created_at	TIMESTAMP	DEFAULT now()	Data utworzenia
updated_at	TIMESTAMP	DEFAULT now()	Data modyfikacji

Tabela 2: Schemat tabeli `bills`

4.1.3 Tabela: bill_events (Zdarzenia legislacyjne)

Kolumna	Typ	Nullable	Opis
id	UUID	PK	Identyfikator
bill_id	UUID	FK → bills	Powiązanie z projektem
event_type	TEXT	NOT NULL	Typ zdarzenia
event_date	TIMESTAMP		Data zdarzenia
description	TEXT		Opis etapu
details	JSONB		Szczegóły (elastyczne dane)
created_at	TIMESTAMP	DEFAULT now()	

Tabela 3: Schemat tabeli `bill_events`

4.1.4 Tabela: profiles (Profile użytkowników)

Kolumna	Typ	Nullable	Opis
id	UUID	PK, FK → auth.users	Identyfikator z Supabase Auth
email	TEXT	UNIQUE	Adres email
full_name	TEXT		Pełna nazwa
avatar_url	TEXT		URL awatara
role	user_role	DEFAULT user	Rola w systemie
is_active	BOOLEAN	DEFAULT true	Czy aktywny (blokada)
last_login	TIMESTAMP		Ostatnie logowanie
created_at	TIMESTAMP	DEFAULT now()	
updated_at	TIMESTAMP	DEFAULT now()	

Tabela 4: Schemat tabeli `profiles`

4.1.5 Dodatkowe tabele

Tabela	Opis
user_alerts	Subskrypcje użytkowników na zmiany w projektach (user_id, bill_id, notify_email, notify_push)
saved_searches	Zapisane wyszukiwania z filtrami (name, query, filters JSONB)
admin_logs	Dziennik aktywności administratorów (action, target_type, target_id, details, ip_address)
system_settings	Ustawienia systemowe (key-value z JSONB i opisem)
consultation_comments	Komentarze konsultacyjne z wątkami (parent_comment_id, content, is_edited)
consultation_comment_reactions	Reakcje na komentarze (like, support, insightful)
notifications	Powiadomienia in-app (type, title, message, is_read, link)

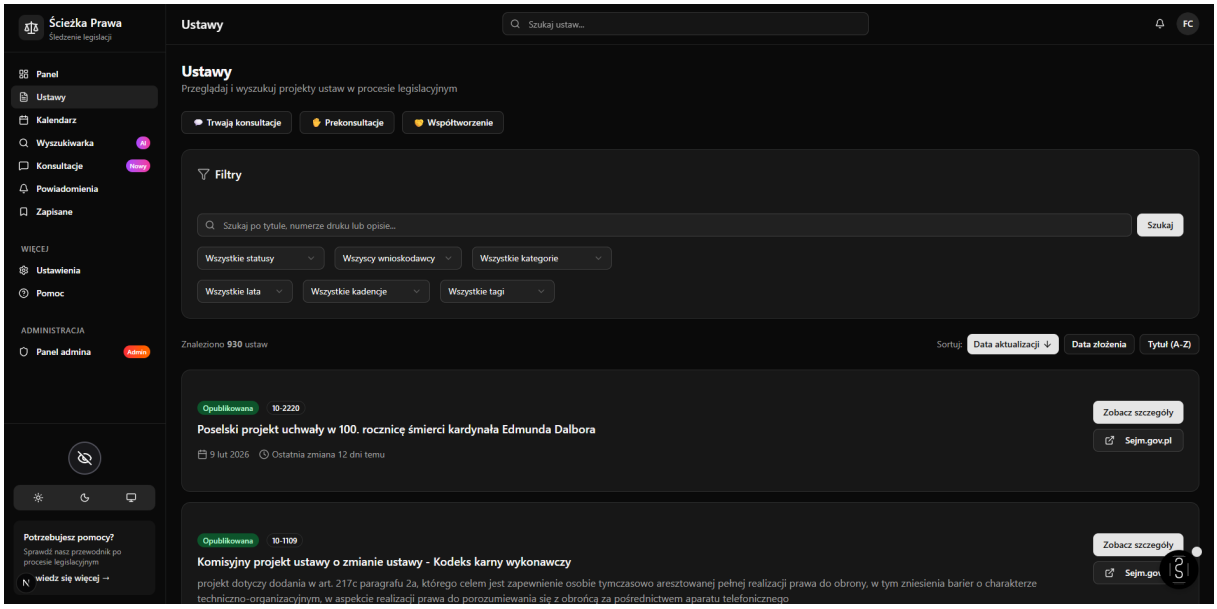
Tabela 5: Dodatkowe tabele w systemie

4.2 Polityki Row-Level Security (RLS)

```sql
-- Bills: Wszyscy czytaja, moderatorzy edytuja
ALTER TABLE bills ENABLE ROW LEVEL SECURITY;
CREATE POLICY "bills_select" ON bills FOR SELECT
  USING (true);  -- Publiczny dostep do odczytu

-- User Alerts: Kazdy zarzadza wlasnymi
CREATE POLICY "alerts_own" ON user_alerts
  FOR ALL USING (auth.uid() = user_id);

-- Profiles: Publiczny odczyt, prywatny zapis
CREATE POLICY "profiles_read" ON profiles FOR SELECT
```

```
  USING (true);
CREATE POLICY "profiles_update" ON profiles FOR UPDATE
  USING (auth.uid() = id);

-- Admin: Service Role Key omija RLS
-- (uzywane do synchronizacji i operacji admina)
```

Kluczowe polityki RLS



Rysunek 4: Lista projektów ustaw z zaawansowanymi filtrami i sortowaniem

5 Integracje API

5.1 API Sejmu RP

Główne źródło danych legislacyjnych. Aplikacja korzysta z publicznego API `api.sejm.gov.pl` dla X kadencji Sejmu.

Endpoint	Metoda	Opis
`/sejm/term10/processes`	GET	Lista wszystkich procesów legislacyjnych
`/sejm/term10/processes/{nr}`	GET	Szczegóły procesu ze stagami
`/sejm/term10/committees/{code}/sittings`	GET	Posiedzenia komisji z URL video

Tabela 6: Endpointy API Sejmu RP

5.1.1 Algorytm mapowania statusów

Logika określania statusu projektu ustawy (`src/lib/api/sejm.ts`):

```
function getStatusFromStages(process): BillStatus {
  // Priorytet 1: Publikacja (ELI lub passed=true)
  if (process.ELI || process.passed) return 'published'
```

```
  // Priorytet 2: Odrzucenie
  if (title.includes('odrzucony')) return 'rejected'

  // Priorytet 3: Analiza ostatniego etapu
  const lastStage = stages[stages.length - 1].stageName.toLowerCase
     ()

  if (lastStage.includes('og oszenie'))    return 'published'
  if (lastStage.includes('prezydent'))     return 'presidential'
  if (lastStage.includes('senat'))         return 'senate'
  if (lastStage.includes('iii czytanie'))  return 'third_reading'
  if (lastStage.includes('ii czytanie'))   return 'second_reading'
  if (lastStage.includes('komisj'))        return 'committee'
  if (lastStage.includes('i czytanie'))    return 'first_reading'
  if (lastStage.includes('skierowanie'))   return 'first_reading'

  return 'submitted'  // Domyslny status
}
```

Algorytm getStatusFromStages()

5.2 Google Gemini 2.0 Flash (AI)

Integracja z modelem `gemini-2.0-flash` do analizy tekstu prawnego.

Tryb	Endpoint	Opis
`simple`	POST /api/ai/simple-language	Tłumaczenie na prosty język polski
`impact`	POST /api/ai/simple-language	Analiza skutków (obywatele, firmy, budżet)
`summary`	POST /api/ai/simple-language	Streszczenie w 3-5 zdaniach
`explain`	POST /api/ai/simple-language	Wyjaśnienie terminologii prawnej
`chat`	POST /api/ai/chat	Asystent konwersacyjny z bazą wiedzy
`search`	POST /api/search/ai	Inteligentna analiza zapytań wyszukiwania

Tabela 7: Tryby AI w systemie

Limity Free Tier Gemini API

- 15 zapytań na minutę
- 1500 zapytań dziennie
- 1 000 000 tokenów dziennie
- Cachowanie odpowiedzi w localStorage (24h)

5.3 YouTube Live Detection

Detekcja transmisji na żywo z kanału Sejmu RP bez klucza API:

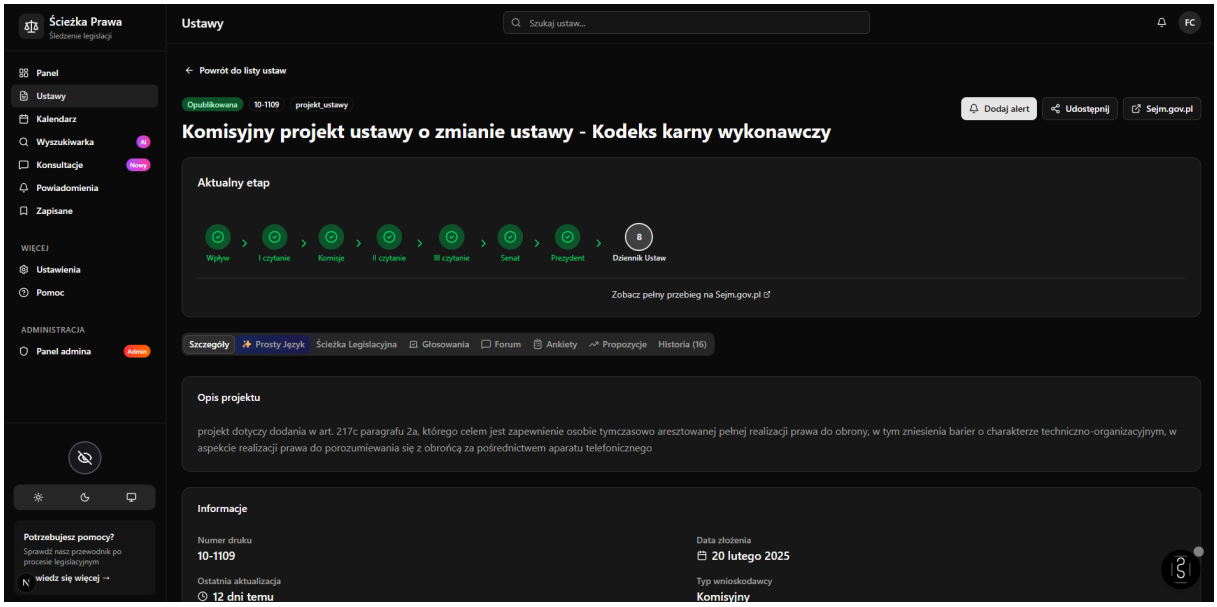
```
// GET /api/youtube-live
// Scraping strony youtube.com/@SejmRP_PL/live
// Wyszukiwanie videoId za pomoca regex
// Cache: 60s revalidation

Response: {
  isLive: boolean,
  videoId: string | null,
  title: string | null,
  channelUrl: "https://www.youtube.com/@SejmRP_PL"
}
```

Mechanizm detekcji YouTube Live

5.4 Resend API (Email)

System powiadomień email z szablonami HTML:

- **Zmiana statusu** — HTML email z badge statusu i linkiem do projektu
- **Raport zbiorczy** — Digest email (dzienny/tygodniowy) z listą zmian
- **Powiadomienie powitalne** — Email po rejestracji nowego użytkownika



Rysunek 5: Widok szczegółów projektu ustawy z osią czasu legislacyjną i analizą AI

6 Endpointy API aplikacji

System udostępnia **34 endpointy** REST API zorganizowane w następujące grupy:

6.1 Endpointy publiczne i użytkownika

Ścieżka	Metody	Auth	Opis
/api/alerts	GET, POST, DELETE	Tak	CRUD alertów użytkownika
/api/calendar	GET	Nie	Zdarzenia Sejmu/Senatu z datami
/api/consultation-comments	GET, POST, PATCH, DEL	Tak*	Forum komentarzy (wątki)
/api/proposals	GET, POST, PATCH, DEL	Tak*	Propozycje poprawek
/api/proposals/vote	POST	Tak	Głosowanie na propozycje
/api/surveys	GET, POST	Tak	Ankiety legislacyjne
/api/surveys/respond	POST	Tak	Odpowiedzi na ankiety
/api/search/ai	POST	Nie	AI analiza wyszukiwania
/api/ai/simple-language	POST	Nie	Tłumaczenie AI (4 tryby)
/api/ai/chat	POST	Nie	Asystent AI konwersacyjny
/api/youtube-live	GET	Nie	Status transmisji YouTube
/api/votings/{nr}	GET	Nie	Wyniki głosowań w Sejmie
/api/notifications	GET, PATCH, DEL	Tak	Zarządzanie powiadomieniami
/api/account/delete	DELETE	Tak	Usunięcie konta

Tabela 8: Endpointy publiczne i użytkownika (* — GET publiczny, modyfikacje wymagają auth)

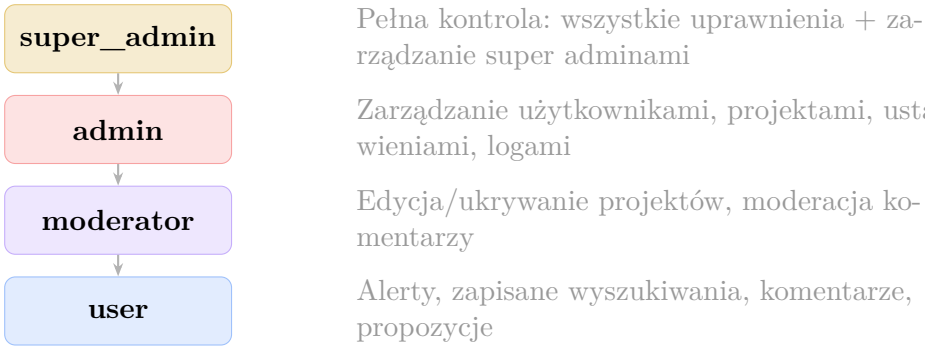
6.2 Endpointy administracyjne

Ścieżka	Metody	Rola	Opis
/api/admin/sync	POST	admin+	Synchronizacja z API Sejmu
/api/admin/sync/{nr}	POST	admin+	Sync konkretnego projektu
/api/admin/sync-rcl	POST	admin+	Synchronizacja z RCL
/api/admin/sync-rcl-enhanced	POST	admin+	Rozszerzona sync RCL
/api/admin/users	GET, PATCH	admin+	Zarządzanie użytkownikami
/api/admin/bills/{id}	PATCH, DELETE	mod+	Edycja/ukrywanie projektów
/api/admin/settings	GET, PUT	admin+	Ustawienia systemowe
/api/admin/logs	GET	admin+	Dziennik aktywności
/api/admin/parse-osr	POST	admin+	Parsowanie OSR (PDF)
/api/notifications/send	POST	system	Wysyłka powiadomień
/api/notifications/process	POST	system	Przetwarzanie kolejki

Tabela 9: Endpointy administracyjne (mod+ = moderator i wyżej, admin+ = admin i wyżej)

7 System uprawnień i bezpieczeństwo

7.1 Hierarchia ról

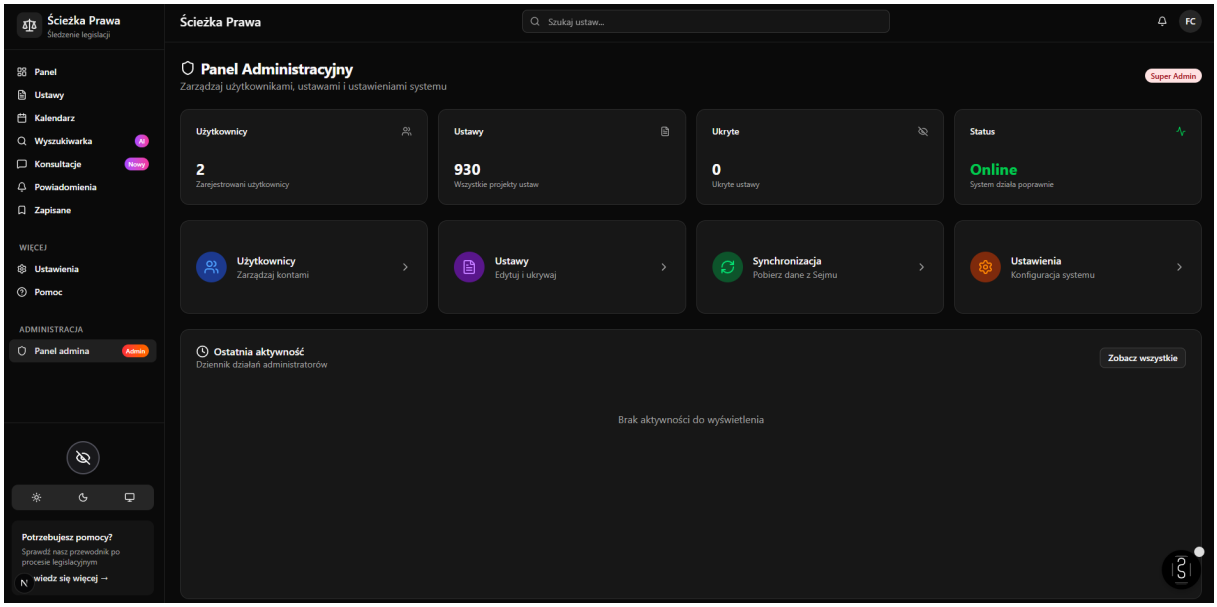


Rysunek 6: Hierarchia ról użytkowników

7.2 Mechanizmy bezpieczeństwa

Mechanizm	Implementacja
Autentykacja	Supabase Auth (email/hasło) z sesjami JWT
Autoryzacja	Row-Level Security (RLS) + sprawdzanie ról w API routes
Service Role	`SUPABASE_SERVICE_ROLE_KEY` do operacji admina (omija RLS)
CSRF	Ochrona przez Next.js middleware + SameSite cookies
XSS	React DOM escaping + walidacja inputów
Rate Limiting	Gemini API: 15 req/min, YouTube: 60s cache
Walidacja	Serverside validation w każdym API route
Logowanie	Tabela `admin_logs` z IP i User-Agent

Tabela 10: Mechanizmy bezpieczeństwa systemu



Rysunek 7: Panel administracyjny z zarządzaniem użytkownikami, synchronizacją i logami

8 Strony i kluczowe funkcjonalności

8.1 Strona główna (Landing Page)

Strona powitalna prezentująca funkcjonalności platformy z interaktywnym demo.

Elementy:

- Banner z informacją o transmisji na żywo Sejmu
- Nagłówek Hero z gradientem i CTA
- Interaktywny mockup dashboardu z prawdziwymi danymi (20 projektów)
- Siatka 3 kluczowych funkcji (real-time, AI, powiadomienia)
- Statystyki (kadencja, posłowie, projekty/miesiąc, monitoring)
- Sekcja z formularzem rejestracji

8.2 Panel główny (Dashboard)

Centralny punkt aplikacji z przeglądem stanu legislacji.

Elementy:

- 4 karty statystyk: łączna liczba projektów, aktywne, opublikowane, alerty
- Banner YouTube Live z auto-odświeżaniem co 2 minuty
- Tabela ostatnio zaktualizowanych projektów (5 newest)

- Rozkład statusów z paskami postępu

8.3 Lista projektów ustaw

Zaawansowane przeglądanie i filtrowanie projektów legislacyjnych.

7 typów filtrów: status, typ wnioskodawcy, kategoria, rok, kadencja, tagi, wyszukiwanie tekstowe.

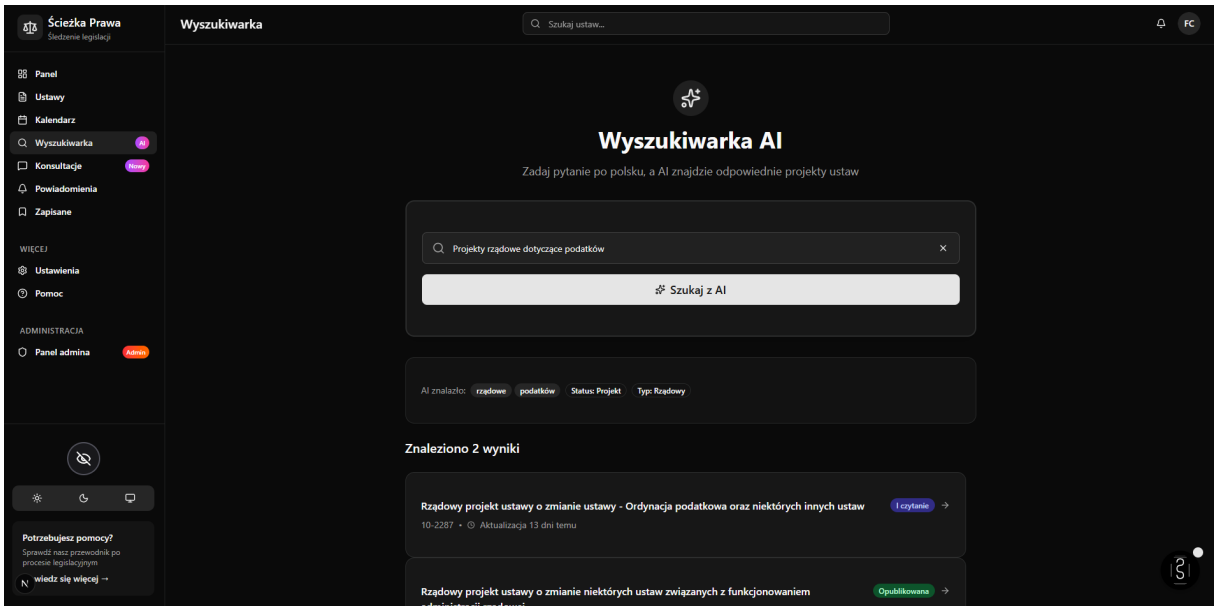
3 opcje sortowania: ostatnia aktualizacja, data wpłynięcia, tytuł.

Paginacja: 10 projektów na stronę z nawigacją prev/next.

8.4 Szczegóły projektu ustawy

Kompleksowy widok z 8 zakładkami:

1. **Ogólne** — opis, ministerstwo, daty, przycisk śledzenia
2. **Ścieżka legislacyjna** — wizualizacja 11-etapowej osi czasu
3. **Głosowania** — wyniki z podziałem na kluby parlamentarne
4. **AI Pomoc** — 4 tryby analizy (prosty język, skutki, streszczenie, wyjaśnienie)
5. **Ocena skutków** — wizualizacja danych OSR
6. **Konsultacje** — forum komentarzy z wątkami i reakcjami
7. **Propozycje** — system propozycji obywatelskich z głosowaniem
8. **Ankiety** — tworzenie i przeglądanie wyników ankiet



Rysunek 8: Wyszukiwarka AI z analizą semantyczną zapytań i sugestiami filtrów

8.5 Wyszukiwarka AI

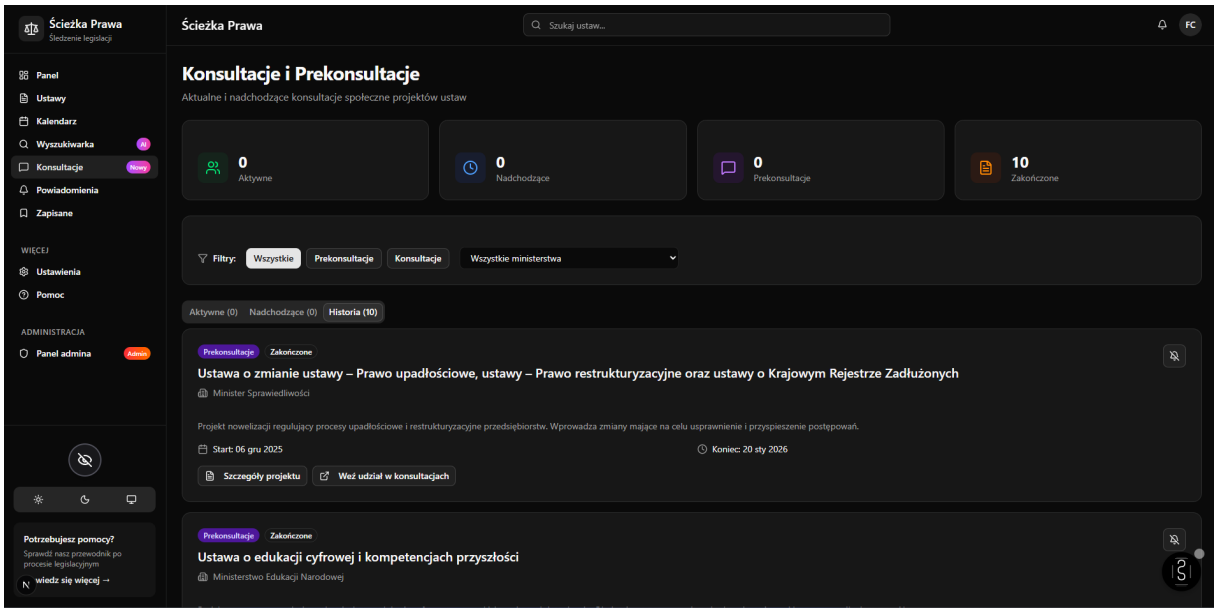
Inteligentne wyszukiwanie projektów z analizą semantyczną:

- Pole wyszukiwania z przetwarzaniem AI zapytania w języku naturalnym
- Automatyczne mapowanie na: kategorię, status, typ wnioskodawcy, ministerstwo
- Zapisywanie wyszukiwań na profilu użytkownika
- Wyniki z kartami statusu i metadanymi

8.6 Kalendarz legislacyjny

- Widok miesiąca z kolorowanymi komórkami zdarzeń
- Kolory: Sejm (czerwony), Senat (niebieski), Komisje (fioletowy)
- Panel szczegółów wybranego dnia
- Lista nadchodzących zdarzeń (7 dni)
- Banner YouTube Live na górze strony
- Przełącznik widoku: miesiąc / lista

8.7 Konsultacje publiczne



Rysunek 9: Strona konsultacji publicznych z aktywnymi i nadchodzącymi konsultacjami

- 3 zakładki: Aktywne, Nadchodzące, Zakończone
- Filtrowanie po ministerstwie i typie (prekonsultacja/konsultacja)
- Karty z datami trwania, liczbą komentarzy i linkami

- 4 karty statystyk: aktywne, nadchodzące, prekonsultacje, zakończone
- Integracja z systemem RCL (Rządowe Centrum Legislacji)

8.8 Panel administracyjny

Panel dostępny dla użytkowników z rolą `moderator+`:

Podstrona	Funkcjonalność
/admin	Dashboard z 4 statystykami, szybkie akcje, ostatnia aktywność
/admin/users	Zarządzanie rolami, blokowanie, wyszukiwanie użytkowników
/admin/bills	Edycja, ukrywanie/odkrywanie projektów
/admin/sync	Ręczna synchronizacja z API Sejmu i RCL
/admin/settings	4 ustawienia: auto-sync, interwał, maintenance, rejestracja
/admin/logs	Dziennik aktywności administracyjnej z filtrami
/admin/test-notification	Testowanie systemu email

Tabela 11: Podstrony panelu administracyjnego

9 Dostępność (WCAG) i UX

9.1 Tryby dostępności

Aplikacja implementuje zaawansowany system dostępności oparty na `AccessibilityProvider`:

Funkcja	Opis
Tryb wysokiego kontrastu (jasny)	Czarny tekst na białym tle, żółte fokusowe obwódki
Tryb wysokiego kontrastu (ciemny)	Biały tekst na czarnym tle, wzmocnione kolory
Tryb odwróconego kontrastu	Inwersja kolorów dla specyficznych potrzeb
Regulacja rozmiaru czcionki	Suwak od 80% do 200% podstawowego rozmiaru
Nawigacja klawiaturą	Pełna obsługa Tab, Enter, Escape, klawisze strzałek
Ciemny/jasny motyw	Automatyczny na podstawie preferencji systemu
Panel dostępności	Sidebar z szybkim dostępem do wszystkich opcji

Tabela 12: Funkcje dostępności WCAG

9.2 Responsywność

- Desktop: pełny sidebar + treść główna
- Tablet: zwijany sidebar z przyciskiem hamburger
- Mobilne: drawer navigation + uproszczony layout
- Wszystkie tabele i karty adaptują się do rozmiaru ekranu

9.3 Język interfejsu

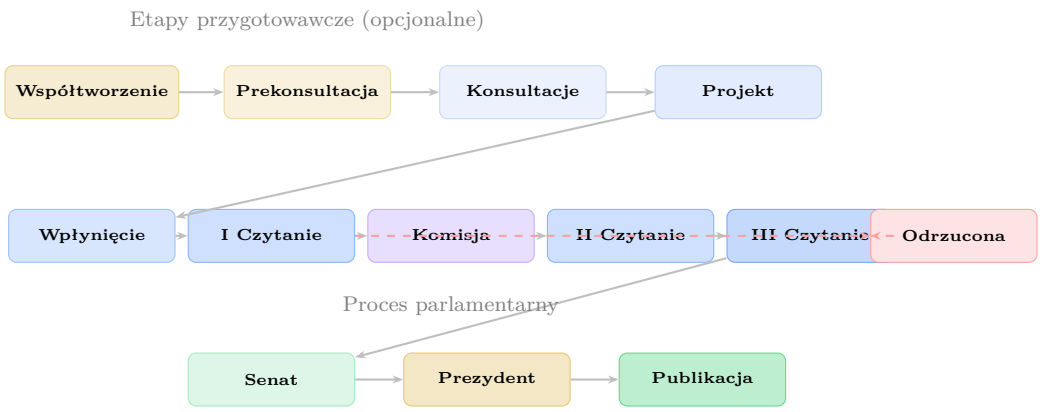
Cały interfejs jest w **języku polskim**:

- Etykiety statusów: *Opublikowana*, *Senat*, *Komisja*, *I/II/III Czytanie*
- Formatowanie dat: `date-fns` z lokalizacją `{locale: pl}`
- Polskie komunikaty błędów, instrukcje i opisy
- Strony polityk: regulamin, prywatność, dostępność, cookies

10 Wizualizacja ścieżki legislacyjnej

Jeden z kluczowych komponentów aplikacji — interaktywna oś czasu procesu legislacyjnego.

10.1 13 statusów projektu ustawy



Rysunek 10: Pełna ścieżka legislacyjna z 13 możliwymi statusami

10.2 Etapy w szczegółach

Status	Klucz	Opis
Współtworzenie	co_creation	Wczesna faza partycypacji obywatelskiej
Prekonsultacja	preconsultation	Wstępne konsultacje przed formalizacją
Konsultacje	consultation	Oficjalne konsultacje publiczne (RCL)
Projekt	draft	Formalny projekt ustawy
Wpłynięcie	submitted	Złożenie projektu w Sejmie
I Czytanie	first_reading	Pierwsze czytanie w Sejmie
Komisja	committee	Prace komisji sejmowej
II Czytanie	second_reading	Drugie czytanie
III Czytanie	third_reading	Trzecie czytanie i głosowanie
Senat	senate	Rozpatrzenie przez Senat
Prezydent	presidential	Oczekiwanie na podpis Prezydenta
Opublikowana	published	Ustawa w mocy (Dz.U., ELI)
Odrzucona	rejected	Projekt odrzucony (na dowolnym etapie)

Tabela 13: Szczegółowy opis 13 statusów legislacyjnych

11 System AI i baza wiedzy

11.1 Architektura AI

System AI oparty jest na trzech filarach:

1. **Google Gemini 2.0 Flash** — główny model do analizy tekstu prawnego
2. **Lokalna baza wiedzy (RAG)** — `src/lib/ai/knowledge-base.ts` z wiedzą o procesie legislacyjnym
3. **Cachowanie localStorage** — 24-godzinny cache odpowiedzi AI po stronie klienta

11.2 4 tryby analizy tekstu

Tryb	Nazwa PL	Opis działania
simple	Prosty Język	Przepisuje tekst prawny językiem zrozumiałym dla laika. Używa prostych słów i krótkich zdań.
impact	Analiza Skutków	Analizuje wpływ na: obywateli, firmy, budżet, środowisko. Podaje daty wejścia w życie i kontrowersje.
summary	Streszczenie	Generuje kompaktowe podsumowanie w 3-5 zdaniach z kluczowymi punktami.
explain	Wyjaśnienie	Szczegółowe definicje terminów prawnych z emoji nagłówkami i przykładami.

Tabela 14: 4 tryby analizy AI dostępne dla każdego projektu ustawy

11.3 Asystent konwersacyjny

Komponent `AIAssistant` — chatbot z bazą wiedzy:

- Priorytetowo: wyszukiwanie w lokalnej bazie wiedzy (RAG)
- Fallback: Google Gemini API gdy brak odpowiedzi w bazie
- Graceful degradation: działa bez klucza API (tylko baza lokalna)
- Kontekst: opisy funkcji aplikacji, etapy legislacyjne, FAQ

11.4 Baza wiedzy RAG

Plik `src/lib/ai/knowledge-base.ts` zawiera:

- Opisy funkcji aplikacji (10 sekcji)
- Definicje 10 etapów procesu legislacyjnego

- Wyjaśnienia ról parlamentarnych
- Opisy typów wnioskodawców
- 8 najczęstszych pytań (FAQ)

12 Komponenty interfejsu użytkownika

12.1 Biblioteka komponentów ShadCN/Radix

Aplikacja korzysta z 26 komponentów ShadCN UI opartych na Radix Primitives:

Komponent	Komponent	Komponent	Komponent
Accordion	Alert	Avatar	Badge
Button	Card	Checkbox	Collapsible
Dialog	Dropdown Menu	Input	Label
Pagination	Popover	Progress	Radio Group
Scroll Area	Select	Separator	Sheet
Skeleton	Slider	Switch	Tabs
Textarea	Tooltip		

Tabela 15: 26 komponentów ShadCN UI w projekcie

12.2 Komponenty domenowe — projekty ustaw

Komponent	Opis
AlertButton	Przycisk subskrypcji zmian w projekcie
LegislativeTimeline	Wizualna oś czasu z 11 etapami i kolorami statusu
LegislativeTrainEnhanced	Rozszerzona wizualizacja w stylu EU-train
SimpleLanguageHelper	AI translator z 4 zakładkami trybów
ImpactAssessmentViewer	Wizualizacja wyników OSR (Ocena Skutków Regulacji)
ImpactAssessmentEnhanced	Rozszerzona wersja z wykresami
ConsultationForum	Forum wątków komentarzy z reakcjami
ProposalList	Lista propozycji obywatelskich z głosowaniem
VotingPieChart	Wykres wyników głosowań z podziałem na kluby
SurveyViewer	Przeglądanie i wypełnianie ankiet

Tabela 16: Kluczowe komponenty domenowe dla projektów ustaw

12.3 Layout i nawigacja

- **DashboardLayout** — główny wrapper z sidebar i headerem
- **Sidebar** — nawigacja z elementami zależnymi od roli (admin = dodatkowe opcje)
- **Header** — pasek wyszukiwania, powiadomienia, menu użytkownika
- **MobileNav** — drawer nawigacyjny na urządzeniach mobilnych
- **AccessibilityButton** — przycisk szybkiego dostępu do panelu dostępności

13 Konfiguracja i wdrożenie

13.1 Zmienne środowiskowe

Zmienna	Wymagana	Opis
NEXT_PUBLIC_SUPABASE_URL	Tak	URL instancji Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Tak	Klucz anonimowy (publiczny)
SUPABASE_SERVICE_ROLE_KEY	Tak	Klucz serwisowy (omija RLS)
GEMINI_API_KEY	Nie*	Klucz Google Gemini 2.0 Flash
RESEND_API_KEY	Nie*	Klucz Resend do emaili

Tabela 17: Zmienne środowiskowe (* — opcjonalne, aplikacja działa bez nich z ograniczoną funkcjonalnością)

13.2 Komendy deweloperskie

```
npm run dev          # Start z Turbopack (port 3000)
npm run build        # Build produkcyjny
npm run start        # Start serwera produkcyjnego
npm run lint         # ESLint
```

Komendy package.json

13.3 Konfiguracja Next.js

```
const nextConfig = {
  // App Router (domyslny w Next.js 16)
  // Turbopack dev server
  // TypeScript strict mode
  // Tailwind CSS 4 integration
}
```

next.config.ts

13.4 Konfiguracja TypeScript

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "strict": true,
    "jsx": "react-jsx",
    "moduleResolution": "bundler",
    "paths": { "@/*": ["./src/*"] }
  }
}
```

tsconfig.json (kluczowe opcje)

13.5 Wdrożenie produkcyjne

Rekomendowane platformy wdrożeniowe:

1. **Vercel** — natywne wsparcie Next.js, automatyczne deployments z Git
2. **Supabase Cloud** — zarządzana baza danych PostgreSQL z RLS
3. **Resend** — darmowy plan 100 emaili/dzień
4. **Google AI Studio** — darmowy tier Gemini 2.0 Flash

14 Podsumowanie

14.1 Statystyki projektu

Metryka	Wartość
Endpointy API	34 route handlers
Tabele bazy danych	8 głównych + typy ENUM
Komponenty UI (ShadCN)	26
Komponenty domenowe	15+
Strony aplikacji	18 (publiczne + auth + admin)
Statusy legislacyjne	13
Tryby AI	6 (4 analiza + chat + search)
Role użytkowników	4 (hierarchiczne)
Polityki RLS	Włączone na wszystkich tabelach
Integracje API	4 (Sejm, YouTube, Gemini, Resend)
Tryby dostępności	4 (kontrast jasny/ciemny/odwrócony + rozmiar)

Tabela 18: Podsumowanie statystyk projektu

14.2 Kluczowe osiągnięcia

- **Pełna automatyzacja** synchronizacji danych z oficjalnego API Sejmu RP
- **Analiza AI** tekstu prawnego z 4 różnymi trybami w czasie rzeczywistym
- **System partycypacji obywatelskiej** — konsultacje, propozycje, ankiety
- **Kompleksowe bezpieczeństwo** — RLS, hierarchia ról, logowanie admina
- **Pełna dostępność WCAG** — tryby kontrastu, regulacja rozmiaru, nawigacja klawiaturą
- **Responsywny design** od desktopa po urządzenia mobilne
- **DX (Developer Experience)** — TypeScript strict, ESLint, komponenty ShadCN

Ścieżka Prawa

Zwycięski Projekt HackNation PL 2025

Autor: **Filip Czerwiński**

Stos: Next.js 16 · React 19 · Supabase · Gemini 2.0 Flash

Przejrzystość prawa w Twoich rękach.